Elyon-Sol: A Deterministic, Fail-Closed Admission Gate for AI Actions with Cryptographic Admissibility Envelopes and Human-in-the-Loop Governance

Justin C. LaPorte — Independent Researcher
ORCID: 0009-0008-3785-3089 · justin@elyon-sol.io · https://elyon-sol.io
Preprint, July 2026 (v1). Implementation snapshot: commit 9645fb8, canon v0.9.8.4, test suite 512/512.
Companion enforcement evidence: Zenodo DOI 10.5281/zenodo.21107731.

Abstract

Autonomous software — agents, tool-callers, and orchestrated pipelines — increasingly performs consequential actions on external systems, yet the controls that decide whether a given action should be permitted lag the capabilities that make actions possible. This paper describes Elyon-Sol, a deterministic, fail-closed HTTP admission gate that answers a narrow, mechanical question before an action runs: is this interaction admissible under a signed, hash-pinned policy? Elyon-Sol is derived from a formal admissibility specification (canon v0.9.8.4) whose model, $G(I) = AC^3 \wedge T^{26} \wedge CCS$, conjoins three invariants — Authority, Coverage, and Continuity. On an admissible decision the gate constructs an admissibility envelope: a content-hashed, Ed25519-signed record binding the decision to the exact canon, manifest, evaluator, request, and condition state under which it was made. A reassertion protocol re-checks that envelope against live state and returns one of three verdicts — reasserted, invalidated, or re-evaluate-required — so a past eligibility is never silently honored after the state it depended on has changed. Above the admissibility core sits a human-in-the-loop governance layer: manifest-classified high-impact actions are held pending a human approver's signed, single-use, separately-keyed grant, with separation of duties enforced through a signed key-record chain. We report development-side evidence: a full test suite of 512 passing tests and a carried-forward enforcement run of 204 calls with zero unauthorized external executions. We are explicit about scope: this is white-box, in-repository evidence; the human-oversight guarantee is deployment-gated, and no external, third-party adversarial validation on a live multi-host surface has yet been performed. That validation is the stated open problem, and an open red-team challenge invites it.

Keywords: admission control, AI governance, human-in-the-loop, deterministic refusal, fail-closed systems, separation of duties, continuity constraints, admissibility envelope, Ed25519, ext-authz.

1. Introduction

The question "can this program do X?" has a mature answer in classical systems security: authenticate the principal, check a policy, permit or deny. The question that autonomous software now raises is subtly different: "should this specific action, with these arguments, against this target, be allowed to happen right now, given the exact policy and world-state in force?" As agents chain tool calls and act with growing autonomy, the interesting failures are not only in what a model says but in what an action-taking system is permitted to do.

Elyon-Sol takes a deliberately narrow position on that question. It does not judge whether an action is wise, aligned, or beneficial; it judges whether an action is admissible — authorized, covered by policy, and consistent with the state the authorization was granted against — and it

refuses everything else, before the action executes. The design is fail-closed: on refusal or on any exception, the downstream target is never called. This trades availability for safety by construction.

This paper makes four contributions:

- A deterministic admission gate derived from a formal specification, realizing three canonical invariants (Section 2) with a fail-closed decision procedure (Section 3).
- A cryptographic admissibility envelope that binds each eligible decision to its full decision context and a reassertion protocol that makes eligibility non-persistent across state change (Sections 4–5).
- A human-in-the-loop governance layer — high-impact hold, human-signed single-use approval grants, provenance-and-role separation of duties, and non-bypass transport — layered above the pinned evaluator without altering it (Section 6).
- An honest, referent-bound evidence practice, reported with its limitations stated as plainly as its results (Sections 7–8), together with an open external-validation challenge (Section 10).

2. The admissibility model

Elyon-Sol implements a canonical specification, fixed at version v0.9.8.4 and pinned by a lockfile (`canon.lock`, SHA-256 `d1c9d187…02b4d7bd`). The canon is treated as an immutable source of record; the implementation realizes it but does not redefine it.

The model evaluates an interaction I against a decision function

$$G(I) = AC^3 \wedge T^{26} \wedge CCS$$

that is admissible only when all three invariants hold:

- Authority (AC^3) — the caller's authority set must satisfy the required authority set for the interaction (canon §§11.3, 11.5).
- Coverage (T^{26}) — the caller's operation set must be covered by the required operation set (canon §§11.4, 11.6).
- Continuity (CCS) — eligibility must be consistent across state transitions: it does not persist once the canon, manifest, or evaluator state it was decided against changes (canon §§12–13).

Required authority and operation sets are not caller-supplied; they are derived from a manifest — a policy document pinned by version and SHA-256. The caller asserts the manifest version and hash it expects; a mismatch is a refusal, not a silent acceptance. This pinning discipline is what makes a decision reproducible and auditable: the decision is a function of named, hashed inputs.

3. The admission gate

The gate is an HTTP boundary. Given a request and the pinned manifest, it returns `ELIGIBLE` only if the authority and coverage conditions are satisfied and the asserted manifest version and hash match; otherwise it returns `REFUSE`. On `ELIGIBLE` the request is forwarded to the target and an admissibility envelope is returned in the response; on `REFUSE` or any exception the target is not called. Determinism and fail-closure are the two load-bearing properties: the same inputs always yield the same decision, and ambiguity always resolves toward refusal.

4. The admissibility envelope

On every eligible decision the gate constructs an admissibility envelope — a canonical record of the inputs and outputs of that single decision, in a form that can be hashed, persisted, and later re-checked. The envelope pins:

- the canon (version and `canon_sha256` read from the lockfile);
- the manifest evaluated against (version and on-disk `manifest_sha256`);
- the request context (the authority set, operation set, caller context, and the caller's expected manifest pins);
- the exact evaluator code (`evaluator_sha256` of the decision logic);
- the individual condition results (Authority, Coverage, and the point-in-time manifest-integrity check);
- a `decision_sha256` computed over the canonical serialization of all of the above.

Canonicalization is deterministic: JSON with sorted keys, no incidental whitespace, and a fixed ASCII discipline, so the decision hash is byte-stable. `decision_sha256` is computed over the envelope minus fields that carry no decision weight — notably the issue timestamp — so the same decision hashes identically regardless of when it was issued.

An envelope may be signed (opt-in) with the gate's issuer key using Ed25519 [3]. The signature covers the decision hash and issuer identity; a bounded validity window (`not_after`) and a per-issuance replay identifier (`decision_id`) are included inside the signed region so an adversary cannot extend a captured envelope's lifetime or replay it, yet excluded from `decision_sha256` so a signed, expiring envelope carries the same decision hash as its unsigned form. Signature validity, freshness, and replay are the verifier's concern and are deliberately separated from admissibility itself.

5. Reassertion and continuity

An admissibility envelope is not a permanent grant. The Continuity invariant holds that an eligibility is valid only relative to the state it was decided against; when that state changes, the decision must be re-checked. The reassertion protocol performs this check. It reads the live pinned hashes — either from local disk or from a trusted, signed published record, which is how a remote target reasserts against a shared state — and evaluates five conditions in strict order, returning the first match:

#	Change detected	Verdict
1	canon hash differs from live	INVALIDATED
2	decision hash fails re-verification (tamper/corruption)	INVALIDATED
3	evaluator hash differs (decision logic moved)	RE-EVALUATE-REQUIRED
4	manifest hash differs (policy moved)	RE-EVALUATE-REQUIRED
5	all hashes match and the decision hash verifies	REASSERTED

The three verdicts have distinct meanings. REASSERTED is the only state in which a past eligibility may be honored without re-evaluation. INVALIDATED is a void decision — the rules

themselves changed, or the artifact was tampered with — and is a hard stop. RE-EVALUATE-REQUIRED is neither honored nor condemned: the logic or policy moved, so the interaction must be evaluated afresh against current state. Ordering is significant; a canon change is reported as invalidated even if the manifest also changed, and tampering takes precedence over a stale hash. Reassertion never mutates the envelope; it reads live state and returns a verdict. The continuity truth-value is derived at reassertion time (true only on REASSERTED), reflecting that on first issuance there is no prior state to compare against.

Together, the envelope and reassertion make eligibility honest under change: a stale "yes" cannot silently persist. Any state shift forces one of three explicit answers — still valid, void, or ask again.

6. Human-in-the-loop governance

Elyon-Sol adds a governance layer above the pinned admissibility core, leaving the hash-pinned evaluator byte-identical. Its purpose is to insert a human where the stakes warrant one, without weakening the deterministic base.

High-impact classification and hold. When the pinned manifest classifies an admitted interaction as high-impact, the gate does not forward it. It returns a `202 PENDING_APPROVAL` terminal state and emits an approval-request identifier. Classification is manifest-derived and fail-closed: a missing or malformed high-impact declaration refuses rather than defaulting to "not high-impact," and an admissible caller cannot self-declare an interaction low-impact.

Human-signed approval grants. A human approver, in a separate process holding a separate private key never resolvable by the gate, signs an approval grant. The gate verifies the grant before forwarding: it must be bound to the exact decision (`decision_sha256`, which transitively binds target, authority and operation sets, context, and manifest pins) and to the specific held request; it must carry a mandatory single-use identifier; it must be fresh (its own expiry); and it must satisfy separation of duties. The grant is then consumed exactly once, before the forward. No code path forwards a high-impact call without a valid, fresh, single-use grant.

Separation of duties via provenance and role. Approver trust is not a key-identifier string comparison. The public keys the gate accepts as approvers are resolved from a signed key-record chain in which an `approver` role is explicit and distinct from the `issuer` role; a key whose signed role is not `approver` is structurally excluded, so a gate-minted self-approval is never honored even if it presents a different key identifier.

Scale and non-bypass. Grant single-use and the pending-approval slot hold across horizontally-scaled instances only with a shared store; a gate that declares itself scaled without one fails closed at startup. A mutual-TLS client-authentication proof shows that a direct call to the target without the gate's client certificate is refused at the TLS handshake, before any application logic; an OPA/Envoy-style external-authorization sidecar [4][5] answers allow/deny by reusing the production verifier rather than re-implementing cryptography; an in-process integration proof shows these mechanisms compose; and a fail-closed startup wiring guard refuses to launch a high-impact deployment that is not wired for safe oversight.

7. Implementation and evidence

The core is a small, dependency-lean Python implementation. Cryptographic enforcement uses Ed25519 signatures over canonical JSON with SHA-256 content hashing; deployment surfaces include the ext-authz sidecar, a mutual-TLS transport proof, shared-store replay and

pending-approval seams, and a JSON-RPC MCP server integration.

Internal consistency. At snapshot commit 9645fb8 the full repository test suite passes 512 of 512, 0 xfailed, up from 419 at the prior published snapshot; the growth is the governance layer and its supporting machinery, which contribute 93 tests (impact classification, approval grants, pep wiring, audit reconciliation, approver provenance/role, shared pending store, mutual-TLS requirement, integration, and the startup wiring guard). Many tests are revert-catchers: they are shown to fail when the specific guard they defend is removed and to pass when it is restored.

Enforcement observation. A carried-forward interception run of 204 calls against a third-party HTTP intake outside the gate's process recorded 102 refusals returning 403 with zero external executions, and 102 eligible calls returning 200 with exactly 102 external executions, each gate-signed, with no duplicate executions. The enforcement path it exercises (admit → sign → forward) is unchanged at this snapshot; the governance layer adds a hold/approval stage above it without altering the default forward.

Adversarial review. The governance core received independent white-box review across three models on separate runs; it found no exploitable defect on a correctly-wired single-process gate and converged on a small set of deployment-posture hardening items, since addressed or scheduled. This is internal convergence evidence, not external validation.

8. Honest scope and limitations

We report development-side, referent-bound evidence — the test suite and runnable proofs — and we do not overstate it.

- No external adversarial validation. No third-party red-team engagement on a real, multi-host public surface has been performed. The interception run is author-driven observation over local transport, not an external penetration test. This is the primary open problem.
- The oversight guarantee is deployment-gated, not certified. The end-to-end property — that the only path to a high-impact execution is through the gate and with a human grant — is claimable only inside a deployment that wires all of the operator-controlled non-bypass layers (inline body binding, mutual-TLS, and network/egress isolation) together with a shared single-use store. That deployment is the operator's to stand up and is not certified here.
- Off-gate callers (A1). A caller that simply does not route through the gate is closeable only by a target-side admission policy plus network isolation, not by the gate alone.
- Root/publisher key compromise is an out-of-band trust-floor concern common to any PKI-rooted system and is not claimed as recovered.
- Distribution. The core implementation is released under AGPL-3.0 (open-core) with the source repository private and access granted on request; a separate administration/tooling SDK is proprietary. AGPL grants every recipient redistribution rights, so access-on-request governs initial, not eventual, visibility.

9. Related work

Elyon-Sol draws on several traditions. The principle that authority should be least and explicit traces to Saltzer and Schroeder [7]; capability-oriented designs make the right-to-act an unforgeable, bound token rather than an ambient permission, which is the spirit of the signed,

decision-bound envelope. Policy-decision engines such as Open Policy Agent [4] externalize authorization from application code; service-mesh external authorization and mutual-TLS identity [5][6] provide the transport-layer non-bypass surface Elyon-Sol targets for deployment. Human-in-the-loop approval and separation of duties are long-standing controls in high-assurance systems; the contribution here is binding a human grant cryptographically to a specific, hash-pinned decision and consuming it exactly once. Relative to emerging AI-agent governance and red-teaming efforts, Elyon-Sol occupies the pre-execution admission niche: a deterministic, attestable boundary on actions rather than a judgment on outputs.

10. Availability, reproducibility, and an open challenge

The canonical model (v0.9.8.4) is published for citation and locked by SHA-256. The implementation is AGPL-3.0 licensed with the source repository private; access is granted on request (`admin@elyon-sol.io` / `justin@elyon-sol.io`). The internal-consistency result is reproducible from the repository at the snapshot commit: verify the canon lock, then run the suite to `512 passed`. Companion enforcement evidence, with the filename-level inventory, is deposited on Zenodo (DOI 10.5281/zenodo.21107731), superseding revisions 2–5 in a documented version chain.

Because external validation is the stated open problem, Elyon-Sol runs a private, invite-only red-team engagement against a live four-node public surface. It is recognition-based rather than a cash bounty: a confirmed break earns permanent, named credit in the project's public verification record, co-credit on the next evidence deposit, and authorship of the fix. Researchers with authorization, protocol, or cryptography backgrounds can request access at `security@elyon-sol.io`; details are at https://elyon-sol.io.

11. Conclusion

Elyon-Sol treats "should this action be allowed?" as a decidable, attestable question, answered before the action runs and refused by default. Its admissibility envelope makes each eligible decision a signed, self-describing artifact; reassertion keeps that decision honest under change; and a human-in-the-loop layer inserts oversight where the stakes require it, without weakening the deterministic base. What remains is the part no author can supply for themselves: an external adversary on a live surface. Until that engagement produces a referent, the guarantees here are what they are stated to be — built, tested, and white-box-reviewed in the repository, and not yet externally certified.

References

[1] Elyon-Sol canonical whitepaper, v0.9.8.4 (locked; `canon.lock` SHA-256 `d1c9d187953eed8145c2d67a98e052415ca2a4c8b722a8011280e21502b4d7bd`). Source of record.

[2] J. C. LaPorte. Elyon-Sol v0.9.8.4 — Enforcement Evidence Addendum (Revision 6). Zenodo, 2026. DOI 10.5281/zenodo.21107731.

[3] S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IRTF, 2017.

[4] Open Policy Agent (OPA), Cloud Native Computing Foundation. Policy-as-code decision engine and the Rego language.

[5] Envoy Proxy. External Authorization (ext_authz) filter. Envoy documentation.

[6] E. Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, IETF, 2018.

[7] J. H. Saltzer and M. D. Schroeder. The Protection of Information in Computer Systems. Proceedings of the IEEE, 63(9), 1975.

[8] RFC 9116. A File Format to Aid in Security Vulnerability Disclosure (security.txt). IETF, 2022.